

Smart Contract Security Audit Report

Generated 2026-09-23 — Automated static analysis (Slither) with AI-assisted commentary

Severity	Count
High	2
Medium	2
Low	2
Informational	5
Optimization	1

Detailed Findings

[High] arbitrary-send-eth — AccessControl.sol

Detector confidence: Medium · Line(s): 22, 23, 24, 25

The `emergencyWithdraw` function in `UnprotectedVault` allows any caller to withdraw an arbitrary amount of ETH to their own address without any access restrictions. Because there is no check verifying that the caller is an authorized party such as an owner or administrator, any external actor can drain the contract's ETH balance entirely. This represents a critical business risk, as the funds held in the vault are fully exposed to theft by any malicious user who interacts with this function.

To remediate this issue, access to `emergencyWithdraw` should be restricted to a trusted role, such as the contract owner, by implementing an `onlyOwner` modifier or an equivalent role-based access control mechanism like OpenZeppelin's `AccessControl` library. Additionally, consider emitting an event upon withdrawal and enforcing a withdrawal limit or time-lock to provide further safeguards against misuse. These controls ensure that only authorized parties can move funds and that all withdrawals are transparent and auditable on-chain.

[High] reentrancy-eth — Reentrancy.sol

Detector confidence: Medium · Line(s): 17, 18, 19, 20, 21, 22, 23, 24

The `withdraw` function in `VulnerableBank` sends Ether to the caller before updating the internal balance record, creating a reentrancy vulnerability. A malicious contract can exploit this by recursively calling `withdraw` during the Ether transfer, draining the contract's funds far beyond the attacker's actual deposit. This could result in a complete loss of all user funds held by the contract.

To remediate this issue, the contract should follow the Checks-Effects-Interactions pattern by updating `balances[msg.sender]` before executing the external Ether transfer, ensuring the state reflects the withdrawal prior to any external call. Alternatively, OpenZeppelin's `ReentrancyGuard` can be applied to the `withdraw` function using the `nonReentrant` modifier to prevent recursive calls entirely. Either approach, or ideally both in combination, will effectively eliminate this attack vector.

[Medium] ERC20-interface — IntegerOverflow.sol

Detector confidence: High · Line(s): 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

The `UncheckedToken` contract implements a `transfer` function that does not conform to the ERC20 standard interface. This means that other contracts, wallets, or decentralized exchanges expecting standard ERC20 behavior may fail to interact with this token correctly, potentially causing loss of funds, failed transactions, or broken integrations. In a production environment, this non-compliance can undermine trust and interoperability across the broader DeFi ecosystem.

To remediate this issue, ensure that the `transfer` function signature matches the ERC20 standard exactly, returning a `bool` value as specified in EIP-20. It is strongly recommended to inherit from a well-audited ERC20 implementation such as OpenZeppelin's `ERC20` contract, which guarantees interface compliance and reduces the risk of subtle bugs. Additionally, consider implementing the `IERC20` interface explicitly so that the compiler enforces correct function signatures at compile time.

[Medium] erc20-interface — TxOriginAuth.sol

Detector confidence: High · Line(s): 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27

The `PhishableWallet` contract implements a `transfer` function with a signature that does not conform to the ERC20 standard interface. This mismatch can cause integration failures with wallets, exchanges, and DeFi protocols that expect standard ERC20 behavior, potentially resulting in lost funds or broken functionality. More critically, a non-standard transfer function may introduce unexpected authorization logic — in this case likely tied to `tx.origin` — that exposes users to phishing attacks where a malicious contract tricks a victim into unknowingly authorizing transfers.

The `transfer` function should be updated to strictly conform to the ERC20 standard as defined in EIP-20, returning a `bool` value and emitting the required `Transfer` event. Any authentication logic relying on `tx.origin` must be replaced with `msg.sender`-based access control, as `tx.origin` should never be used for authorization. Adopting a well-audited ERC20 base implementation, such as OpenZeppelin's `ERC20` contract, is strongly recommended to ensure correctness and prevent future interface deviations.

[Low] missing-zero-check — AccessControl.sol

Detector confidence: Medium · Line(s): 17, 18

The `setOwner` function in `AccessControl.sol` does not verify that the new owner address provided is non-zero before assigning it. This means an administrator could accidentally call the function with a zero address (e.g., due to a scripting error or user mistake), permanently locking ownership of the vault and rendering any owner-restricted functions inaccessible. Since ownership controls critical administrative privileges, such a mistake could result in an irreversible loss of contract control.

To remediate this issue, add an explicit zero-address check at the beginning of the `setOwner` function, reverting the transaction if `newOwner` equals `address(0)`. This can be implemented with a simple `require(newOwner != address(0), "Invalid address: zero address")` statement before the assignment. Additionally, consider emitting an event upon ownership transfer to provide an auditable on-chain record of administrative changes.

[Low] missing-zero-check — TxOriginAuth.sol

Detector confidence: Medium · Line(s): 22, 23

The `transfer` function in `PhishableWallet` does not validate that the destination address `to` is non-zero before executing the transfer. If a caller accidentally or maliciously supplies a zero address, funds sent to that address are permanently and irrecoverably lost, as no one controls the zero address. While this may appear to be an unlikely scenario, the absence of a guard introduces unnecessary financial risk, particularly in environments where automated scripts or integrations might pass uninitialized address variables.

To remediate this issue, add an explicit zero-address check at the beginning of the `transfer` function, reverting the transaction if `to` is equal to `address(0)`. This can be implemented with a simple `require(to != address(0), "Invalid recipient address")` statement before the transfer is executed. Adopting this as a standard defensive practice across all functions that accept address parameters will reduce the risk of accidental fund loss.

[Informational] solc-version — AccessControl.sol

Detector confidence: High · Line(s): 2

The project's `AccessControl.sol` contract specifies a Solidity compiler version constraint of `^0.8.19`, which is associated with several known compiler bugs, including issues related to verbatim block deduplication, argument evaluation order during inlining, and missing side effects on selector access. While these bugs may not be directly exploitable in every contract, they can lead to unexpected or incorrect compiled bytecode, potentially introducing subtle vulnerabilities that are difficult to detect through standard testing. In a security-sensitive contract like access control, even minor compiler-induced misbehavior could have serious consequences for the integrity of the system.

It is recommended to update the Solidity version constraint to a more recent, stable release — such as `^0.8.24` or the latest patched version — that does not carry these known issues. The pragma should be updated consistently across all contracts in the project to ensure a uniform and safe compilation environment. After updating, the full test suite should be re-executed to confirm that no behavioral changes were introduced by the compiler upgrade.

[Informational] solc-version — IntegerOverflow.sol

Detector confidence: High · Line(s): 2

The project uses Solidity compiler version `^0.8.19`, which is associated with several known bugs including `VerbatimInvalidDeduplication`, `FullInlinerNonExpressionSplitArgumentEvaluationOrder`, and `MissingSideEffectsOnSelectorAccess`. These bugs can lead to incorrect code generation or unexpected behavior during compilation, potentially introducing vulnerabilities that are difficult to detect through standard code review. While the likelihood of impact depends on specific code patterns, using a compiler version with known issues introduces unnecessary risk to the integrity of the deployed contracts.

It is recommended to update the Solidity pragma to a more recent, stable compiler version that does not carry these known issues, such as `^0.8.25` or the latest patched release available at the time of deployment. The pragma statement should be updated from `^0.8.19` to a specific, fixed version (e.g., `pragma solidity 0.8.25;`) to ensure deterministic compilation behavior and eliminate exposure to the identified bugs. Always consult the official Solidity bug disclosure list at <https://solidity.readthedocs.io/en/latest/bugs.html> before selecting a compiler version.

[Informational] solc-version — Reentrancy.sol

Detector confidence: High · Line(s): 2

The project uses Solidity compiler version `^0.8.19`, which is associated with several known compiler-level bugs, including issues with verbatim block deduplication, incorrect argument evaluation ordering during inlining, and missing side effects on selector access. While these bugs may not directly manifest as exploitable vulnerabilities in every contract, they can introduce subtle, hard-to-detect misbehaviors in compiled bytecode that deviate from the intended source code logic. This creates risk that the deployed contract may not behave as expected, potentially undermining business logic or security guarantees.

It is recommended to update the Solidity compiler version to the latest stable release (currently `0.8.26` or higher) to ensure all known compiler bugs are patched. Replace the version pragma `^0.8.19` with a fixed, up-to-date version such as `pragma solidity 0.8.26;` to prevent accidental compilation with older, vulnerable compiler versions. After upgrading, the contract should be fully recompiled, retested, and redeployed to ensure correctness under the new compiler version.

[Informational] low-level-calls — Reentrancy.sol

Detector confidence: High · Line(s): 17, 18, 19, 20, 21, 22, 23, 24

The `withdraw` function in `VulnerableBank` uses a low-level `.call()` to transfer Ether to the caller, which provides minimal safety guarantees and bypasses Solidity's built-in error handling. This pattern is particularly dangerous in the context of a withdrawal function because it forwards all available gas to the recipient, creating a direct pathway for reentrancy attacks where a malicious contract could repeatedly call `withdraw` before the balance is updated. From a business perspective, this could result in complete drainage of contract funds and significant financial loss to users and the protocol.

It is strongly recommended to follow the Checks-Effects-Interactions pattern by updating the user's balance in storage before making any external call, ensuring that even if a reentrant call occurs, the balance will already reflect zero. Additionally, consider implementing OpenZeppelin's `ReentrancyGuard` and using the `nonReentrant` modifier on the `withdraw` function as a secondary defense layer. If a low-level call must be retained, ensure the return value of `success` is properly checked and the transaction is reverted on failure to prevent silent errors.

[Informational] solc-version — TxOriginAuth.sol

Detector confidence: High · Line(s): 2

The project uses Solidity compiler version `^0.8.19`, which is associated with several known compiler bugs, including issues related to invalid deduplication of verbatim blocks, incorrect argument evaluation order during inlining, and missing side effects on selector access. While these bugs may not be directly exploitable in every contract, they can introduce subtle and unpredictable behavior at the compiler level, potentially leading to incorrect bytecode generation that is difficult to detect through standard testing or review.

It is recommended to upgrade the Solidity compiler version to the latest stable release (e.g., `^0.8.26` or the most recent patch at time of deployment) to benefit from bug fixes and security improvements. Update the pragma statement in `TxOriginAuth.sol` accordingly and re-run the full test suite to confirm that contract behavior remains consistent after the upgrade.

[Optimization] immutable-states — TxOriginAuth.sol

Detector confidence: High · Line(s): 10

The `owner` variable in the `PhishableWallet` contract is set once during deployment in the constructor and never modified afterward, yet it is declared as a regular storage variable rather than an `immutable` one. This means every read of `owner` incurs an unnecessary SLOAD operation, consuming more gas than required and increasing the cost of any function that references it. While this does not introduce a direct security vulnerability, it represents an avoidable inefficiency that inflates transaction costs over the contract's lifetime.

To remediate this issue, the `owner` variable should be declared with the `immutable` keyword, as in `address immutable owner`. This instructs the Solidity compiler to embed the value directly into the contract bytecode at deployment time, replacing costly storage reads with cheap bytecode reads. Making this change will reduce gas consumption for all operations that check the owner, improving overall contract efficiency at no cost to functionality or security.